Plan de proiect: partiționarea proceselor cu QAOA

Oct 22, 2026 · @Ana

La ce proiect lucrezi?

Proiectul compară, pe calculatoare cuantice IBM reale, un algoritm cuantic (QAOA) cu algoritmi clasici pe o problemă de sisteme distribuite: împărțirea proceselor unui program paralel între două mașini.

În o frază (pentru oricine): „Testez dacă un calculator cuantic real poate rezolva o problemă din sisteme distribuite, cum să împarți procesele între mașini ca să comunice cât mai puțin, și compar rezultatele cu algoritmii clasici."

În 30 de secunde (interviu, networking): „Când rulezi un program paralel pe mai multe mașini, procesele care comunică mult ar trebui să stea pe aceeași mașină, altfel plătești în latență și bandă. Asta e o problemă de partiționare a grafurilor, NP-hard. Am formulat-o ca problemă de optimizare cuantică, am rezolvat-o cu QAOA pe simulator și pe procesoare IBM reale, și am comparat rezultatele cu forța brută, o euristică greedy și sampling aleatoriu. Partea interesantă nu e dacă câștigă quantum, ci *unde și de ce* se rupe pe hardware-ul de azi: de exemplu, cum transformă constrângerea de echilibru un graf rar într-un circuit dens și zgomotos."

Varianta tehnică (cineva care știe quantum): „Minimum graph bisection pe grafuri cu topologii de comunicare tipice (inel, grilă, 3-regulat, aleatoriu), formulat ca Hamiltonian Ising cu penalizare pătratică de echilibru. QAOA cu p = 1–3, parametri optimizați pe statevector și transferați pe QPU. Măsor raportul de aproximare normalizat față de baseline-ul aleatoriu, probabilitatea optimului și efectul densității Hamiltonianului asupra adâncimii după transpilare, comparând Max-Cut (rar) cu bisecția penalizată (dens) pe aceleași grafuri."

Bullet pentru CV (îl completezi cu cifrele tale la final):

- *Quantum vs. classical graph partitioning for distributed systems* — Formulated process-placement as a QUBO/Ising problem and solved it with QAOA (Qiskit) on IBM quantum hardware; benchmarked against brute-force, greedy and random baselines on [N] graph instances up to [X] qubits; showed that [principala ta concluzie, cu un număr].

Motivația

Proiectul răspunde cu date proprii la o întrebare concretă: cât de bine rezolvă un calculator cuantic de azi o problemă reală din sisteme distribuite, față de metodele clasice?

Problema de sisteme. Un program MPI cu N procese rulează pe două mașini. Comunicarea între procese de pe aceeași mașină e ieftină (memorie partajată). Între mașini trece prin rețea și costă latență și bandă. Vrei deci ca procesele care comunică mult să stea împreună, dar și ca fiecare mașină să primească jumătate din procese, altfel una e supraîncărcată. Asta e **bisecția minimă a unui graf**: o problemă NP-hard, rezolvată în practică cu euristici (METIS, Kernighan-Lin).

De ce quantum. Problemele de optimizare combinatorială se pot scrie ca „energia" unui sistem de qubiți. QAOA e un algoritm hibrid care caută stările de energie mică, deci soluții bune. E unul dintre cei mai studiați candidați pentru calculatoarele cuantice de azi, zgomotoase și de dimensiuni medii.

Ce ne așteptăm, onest. La 6–14 qubiți, forța brută găsește optimul instantaneu. QAOA nu va bate metodele clasice. Scopul nu e să demonstrăm un avantaj, ci să măsurăm:

- cât de aproape ajunge QAOA de optim, ideal și pe hardware real;
- cât pierde din cauza zgomotului;
- ce caracteristici ale problemei (tipul grafului, densitatea Hamiltonianului) decid dacă merge sau nu.

De ce e valoros. Majoritatea proiectelor studențești rulează un tutorial QAOA și raportează „a găsit soluția". Proiectul ăsta are o problemă motivată din sisteme, baseline-uri clasice și aleatorii corecte, predicții făcute înainte de a rula pe QPU și o analiză care explică rezultatele. Asta îl face credibil la un interviu de systems, de quantum sau de data.

Ce înveți concret: formularea problemelor de optimizare ca QUBO/Ising, construcția și rularea circuitelor variaționale, cum arată hardware-ul cuantic real (topologie, erori, transpilare), metodologie experimentală și interpretarea datelor zgomotoase.

Întrebări de cercetare și scope

Proiectul răspunde la cinci întrebări; fiecare grafic și fiecare tabel din analiza finală trebuie să servească cel puțin uneia dintre ele.

Cod	Întrebare	Ipoteza ta inițială (o rescrii în Etapa 5)
Î1	Pe simulator ideal, cât de aproape de optim ajunge QAOA, în funcție de p și de tipul grafului?	Crește cu p; mai bine pe grafuri regulate
Î2	Cât pierde QAOA pe QPU real față de simulatorul ideal, în funcție de numărul de qubiți?	Pierderea crește cu numărul de qubiți
Î3	Face QAOA pe QPU mai bine decât sampling-ul aleatoriu cu același număr de încercări?	Da la grafuri mici, nu la cele mari
Î4	Cum se compară QAOA cu forța brută și greedy, ca calitate a soluției?	Clasicul găsește optimul; QAOA rămâne în urmă
Î5	Cât costă constrângerea de echilibru pe hardware? (Max-Cut rar vs. bisecție densă, pe aceleași grafuri)	Bisecția are circuite mult mai adânci și rezultate mai slabe

Ipotezele sunt intenționat simple. Le scrii în cuvintele tale înainte să ai date și le compari cu rezultatele la final: ce ai ghicit și ce te-a surprins e o parte valoroasă a concluziilor.

Intră în proiect:

- grafuri generate cu `networkx`, 6–14 noduri, patru tipuri (inel, grilă, 3-regulat, aleatoriu) plus graful complet ca test de sanitate;
- două probleme pe aceleași grafuri: Max-Cut (din tutorial) și bisecția minimă (problema ta de sisteme);
- solvere clasice: forță brută și greedy cu restarturi;
- QAOA cu p = 1, 2, 3 pe simulator ideal, simulator cu zgomot și QPU (pe QPU doar un subset);
- analiză statistică și README.

Nu intră (posibile extensii, doar după ce totul e gata):

- date reale de comunicare din programe MPI;
- rularea efectivă a programelor pe partițiile găsite;
- mixeri QAOA avansați care respectă constrângerea fără penalizare;
- tehnici avansate de reducere a erorilor;

- împărțirea în mai mult de două grupuri.

Cum folosești planul

Proiectul are nouă etape, aproximativ 33 de ore în total; fiecare etapă se termină cu ceva care funcționează și pe care îl poți arăta.

Fiecare etapă are aceeași structură:

- **Teorie de învățat:** conceptele de care ai nevoie, explicate pe scurt, plus unde le aprofundezi;
- **Verifică-te:** întrebări la care trebuie să poți răspunde fără notițe înainte să treci la task-uri;
- **Task-uri:** pașii concreți, de bifat;
- **Livrabil și criteriu de terminare:** cum știi sigur că etapa e gata.

Regula de lucru: nu treci la etapa următoare până nu poți explica cu voce tare ce ai făcut în cea curentă și de ce. Când lucrăm împreună, eu explic, răspund la întrebări și te ajut la debugging, dar codul îl scrii tu și deciziile le iei tu. Ținem un fișier `JURNAL.md` în repo, unde notezi la fiecare sesiune ce ai făcut, ce nu a mers și ce ai învățat; la final, din el scrii README-ul.

Etapele și timpul estimat:

1. Etapa 0 — Setup și primul circuit pe QPU (2h)
2. Etapa 1 — Fundamente cuantice minime (3h)
3. Etapa 2 — Problema clasică și solverele clasice (5h)
4. Etapa 3 — Din graf în Hamiltonian (4h)
5. Etapa 4 — QAOA pe simulator (5h)
6. Etapa 5 — Transpilare, hardware și predicții (3h)
7. Etapa 6 — Rulările pe QPU (3h)
8. Etapa 7 — Analiza și interpretarea datelor (5h)
9. Etapa 8 — Documentare și prezentare (3h)

Orele sunt orientative. Dacă o etapă de teorie îți ia dublu, e în regulă: înțelegerea de acolo îți scurtează etapele de cod.

Etapa 0 — Setup și primul circuit pe QPU (2h)

La finalul etapei ai un mediu de lucru funcțional și ai rulat un circuit Bell atât pe simulator, cât și pe un QPU real.

Teorie de învățat

- **Qiskit patterns:** orice program Qiskit are patru pași. *Map* (problema devine circuit), *Optimize* (circuitul e transpilat pentru hardware), *Execute* (rulează pe simulator sau QPU), *Post-process* (interpretezi rezultatele). Tot proiectul urmează structura asta.
- **Primitivele:** `Sampler` îți dă bitstring-uri măsurate și frecvența lor („ce soluții au ieșit”). `Estimator` îți dă valoarea medie a unei observabile („ce energie are starea”). În QAOA folosești Estimator la optimizare și Sampler la final.
- **Shots:** un circuit cuantic e probabilistic, deci îl rulezi de multe ori (de exemplu 4.000). Fiecare rulare e un shot.
- **Simulator vs. QPU:** simulatorul calculează exact sau imită zgomotul, local, gratuit. QPU-ul e hardware real, cu coadă de așteptare, zgomot și buget de timp.
- **Contul IBM:** token API + instanță. QPU time se consumă doar cât rulează efectiv circuitul, nu cât stai la coadă.

Unde: pagina „Get started” și ghidul despre primitive din documentația IBM Quantum; cursul „Basics of quantum information” de pe IBM Quantum Learning, primele lecții.

Verifică-te

- Care e diferența dintre Sampler și Estimator și când îl folosești pe fiecare?
- De ce nu e suficient un singur shot?
- Ce înseamnă transpilarea și de ce e nevoie de ea?

Task-uri

- [x] ~~Creează repo-ul Git cu structura din secțiunea „Structura repo-ului” (la final) și fișierul `JURNAL.md`~~
- [x] ~~Creează un mediu virtual Python (3.10 sau mai nou) și instalează `qiskit`, `qiskit-ibm-runtime`, `qiskit-aer`, `networkx`, `numpy`, `scipy`, `pandas`, `matplotlib`, `jupyter`~~
- [x] ~~Notează versiunile exacte în `requirements.txt` (`pip freeze`), ca proiectul să fie reproductibil~~

- [x] ~~Salvează contul IBM local cu `QiskitRuntimeService.save_account(...)` și verifică ce backend-uri vezi~~
- [x] ~~Construiește un circuit Bell (H pe qubitul 0, CNOT 0→1, măsurare) și rulează-l cu Sampler pe simulatorul Aer~~
- [x] ~~Rulează același circuit pe un QPU real, 1.000 de shot-uri; notează în jurnal ID-ul job-ului, backend-ul și timpul de QPU consumat~~
- [x] ~~Compară cele două histograme: ce rezultate „imposibile” (01, 10) apar pe QPU și în ce procent?~~

Livrabil și criteriu de terminare

Notebook-ul `00_setup.ipynb` cu cele două histograme alăturate și o frază de interpretare. Gata când poți explica de ce apar 01 și 10 pe QPU, dar nu și pe simulator.

Etapa 1 — Fundamente cuantice minime (3h)

La finalul etapei înțelegi de ce o funcție de cost pe bitstring-uri se poate scrie cu operatori Z, ideea pe care se sprijină tot QAOA.

Teorie de învățat

- **Qubit și măsurare:** un qubit e o combinație $a|0\rangle + b|1\rangle$. La măsurare obții 0 cu probabilitatea $|a|^2$ și 1 cu $|b|^2$. Poarta Hadamard (H) pe $|0\rangle$ dă probabilitate ½ pentru fiecare.
- **Mai mulți qubiți:** n qubiți au 2^n stări de bază, câte una pentru fiecare bitstring. O stare generală e o suprapunere a tuturor, iar măsurarea „alege” un bitstring. De aici legătura cu optimizarea: fiecare bitstring e o posibilă împărțire a proceselor.
- **Operatorul Z:** $Z|0\rangle = +|0\rangle$ și $Z|1\rangle = -|1\rangle$. Deci Z „citește” bitul ca spin: $z = +1$ pentru bitul 0, $z = -1$ pentru bitul 1, adică $z = 1 - 2x$.
- **Produsul Z_iZ_j:** dă +1 dacă biții i și j sunt egali și −1 dacă diferă. Asta e cheia: $(1 - Z_iZ_j)/2$ valorează 1 exact când i și j sunt în grupuri diferite, adică muchia (i, j) e tăiată.
- **Hamiltonian diagonal = funcție de cost:** o sumă de termeni Z și ZZ e o matrice diagonală, iar valoarea de pe diagonală pentru fiecare bitstring e costul acelui bitstring. „Energia” unei stări = costul mediu al bitstring-urilor pe care le dă la măsurare.
- **Porțile din QAOA:** $RZZ(\theta)$ aplică o fază care depinde de egalitatea a doi biți, iar $RX(\theta)$ rotește un qubit și amestecă 0 cu 1. QAOA alternează aceste două tipuri de porți.

- **Capcana de ordine din Qiskit:** Qiskit numerotează biții de la dreapta. În bitstring-ul `'011'`, qubitul 0 este ultimul caracter. La fel în `SparsePauliOp`: eticheta `'IZ'` înseamnă Z pe qubitul 0. Cele mai multe bug-uri din proiectele QAOA vin de aici.

Unde: IBM Quantum Learning, „Basics of quantum information” (lecțiile despre sisteme simple și multiple) și pagina despre ordinea biților din documentația Qiskit.

Verifică-te

- Ce valoare are ⟨Z⟩ pentru starea |0⟩? Dar pentru H|0⟩? De ce?
- Pentru bitstring-ul 0110 (citit ca în Qiskit), ce valoare au Z_0Z_1 și Z_1Z_2?
- De ce $(1 - Z_iZ_j)/2$ numără muchiile tăiate?
- Ce înseamnă eticheta `'ZIIZ'` într-un `SparsePauliOp` pe 4 qubiți?

Task-uri

- [x] ~~Pe hârtie: pentru un triunghi (3 noduri, 3 muchii), scrie suma $\Sigma(1 - Z_iZ_j)/2$ și calculează-i valoarea pentru toate cele 8 bitstring-uri~~
- [x] ~~În cod: construiește același operator ca `SparsePauliOp` și compară diagonala lui (via `to_matrix()`) cu tabelul tău de pe hârtie~~
- [x] ~~Scrie funcția `bitstring_to_partition(s)` care respectă ordinea Qiskit și testeaz-o pe 3 exemple scrise de mână~~
- [x] ~~Pregătește starea H⊗H⊗H, calculează ⟨H_C⟩ cu `StatevectorEstimator` și verifică că dă media costurilor celor 8 bitstring-uri~~
- [x] ~~Notează în jurnal, în cuvintele tale, legătura „Hamiltonian ↔ funcție de cost”~~

Livrabil și criteriu de terminare

Notebook-ul `01_fundamente.ipynb`. Gata când tabelul de pe hârtie, diagonala matricei și valoarea din Estimator se potrivesc exact.

Etapa 2 — Problema clasică și solverele clasice (5h)

La finalul etapei ai setul fix de grafuri al proiectului și, pentru fiecare, soluția optimă, rezultatul greedy și distribuția costurilor aleatorii: referința pentru tot restul.

Teorie de învățat

- **Tăietura unui graf:** împarți nodurile în două grupuri; tăietura = numărul (sau greutatea) muchiilor cu capetele în grupuri diferite.
- **Max-Cut:** maximizezi tăietura, fără constrângeri. E problema din tutorialul IBM.
- **Bisecția minimă:** minimizezi tăietura, cu grupuri egale (n/2 + n/2). E problema ta: puțină comunicare între mașini, încărcare echilibrată. Fără constrângerea de echilibru, răspunsul ar fi trivial: toate procesele pe o mașină, tăietură 0.
- **NP-hard:** nu se cunoaște niciun algoritm care să găsească garantat optimul în timp polinomial. La n mic, forța brută merge fără probleme: Max-Cut are 2^{n-1} împărțiri distincte, iar bisecția C(n, n/2)/2, adică 1.716 la n = 14.
- **Simetria:** bitstring-ul și complementul lui (toți biții inversați) descriu aceeași împărțire. De aici împărțirea la 2 de mai sus. Contează și la interpretarea rezultatelor QAOA.
- **Degenerarea:** o problemă poate avea mai multe soluții optime diferite. Când calculezi „probabilitatea de a găsi optimul”, numeri toate bitstring-urile optime, deci trebuie să știi câte sunt.
- **Căutarea locală (greedy):** pornești de la o soluție și faci mutări mici care scad costul până nu mai există. Pentru bisecție, mutarea e un *swap* (schimbi două noduri între grupuri, ca echilibrul să rămână). Se poate bloca într-un optim local, de aceea se folosesc restarturi. E ideea din spatele algoritmului Kernighan-Lin.

Unde: orice curs de algoritmi pentru NP-hard și căutare locală; articolul Kernighan-Lin (1970) pentru context; pentru formulare, secțiunea despre graph partitioning din Lucas, „Ising formulations of many NP problems” (2014).

Verifică-te

- De ce nu putem folosi Max-Cut direct pentru problema de sisteme?
- Câte bisecții distincte are un graf cu 10 noduri?
- Ce e un optim local și de ce ajută restarturile?
- Care e bisecția minimă pentru un graf complet cu 8 noduri și câte soluții optime are?

Task-uri

- [x] ~~Scrie generatoarele de grafuri, fiecare cu seed fix: inel, grilă (2×3, 2×4, 2×5, 3×4, 2×7), 3-regulat aleatoriu, aleatoriu Erdős–Rényi (p = 0,4, verifici că e conex) și complet (test de sanitate)~~

- [x] ~~Generează setul de instanțe pentru $n \in \{6, 8, 10, 12, 14\}$ și salvează-l în `data/instances.json` (listă de muchii + tip + seed); setul nu se mai schimbă după asta~~
- [x] ~~Implementează `cut_value(graph, partition)` și testează-o pe 3 cazuri calculate de mână~~
- [x] ~~Forță brută pentru Max-Cut și pentru bisecție: valoarea optimă și **toate** soluțiile optime~~
- [x] ~~Greedy pentru bisecție (swap-uri, 20 de restarturi) și pentru Max-Cut (mutări de un nod)~~
- [x] ~~Pentru fiecare instanță, calculează media și deviația standard a costului pe toate bisecțiile echilibrate (sau pe 10.000 eșantionate), referința „aleatoriu" din etapa de analiză~~
- [x] ~~Construiește tabelul de referință: instanță, n, muchii, optim, număr de optime, greedy, cost aleatoriu mediu~~
- [x] ~~Graficul de verificare: histograma costurilor tuturor bisecțiilor pentru 2–3 instanțe, cu optimul marcat~~

Livrabil și criteriu de terminare

Modulul `src/classical.py`, fișierul `data/reference.csv` și notebook-ul `02_clasic.ipynb`. Gata când testele trec, graful complet dă rezultatul calculat de mână și poți explica de ce greedy ratează optimul pe instanța unde îl ratează (dacă există una).

Etapa 3 — Din graf în Hamiltonian (4h)

La finalul etapei ai, pentru fiecare instanță, doi Hamiltonieni verificați exhaustiv: unul pentru Max-Cut și unul pentru bisecția minimă. Aici e contribuția ta teoretică principală.

Teorie de învățat

- **QUBO:** o funcție de cost pătratică în variabile binare $x_i \in \{0, 1\}$. Multe probleme NP-hard se scriu așa.
- **De la QUBO la Ising:** substitui $x_i = (1 - z_i)/2$, cu $z_i \in \{+1, -1\}$, apoi înlocuiești fiecare z_i cu operatorul Z_i. Rezultatul e un Hamiltonian cu termeni Z și ZZ.
- **Max-Cut:** tăietura e numărul de muchii cu biți diferiți, deci minimizezi suma Z_iZ_j pe muchii:

```
H_{MC} = \sum_{(i,j) \in E} Z_i Z_j
```

- **Bisecția minimă cu penalizare:** costul tăieturii plus o penalizare care e zero doar când grupurile sunt egale ($\Sigma z_i = 0$ exact atunci):

```
H_{B} = \sum_{(i,j) \in E} \frac{1 - Z_i Z_j}{2} + A \Big( \sum_{i} Z_i \Big)^2
```

- **Consecința importantă a penalizării:** dezvoltând pătratul, $(\Sigma Z_i)^2 = n + 2\cdot\Sigma\, Z_iZ_j$ pe **toate** perechile i < j. Deci Hamiltonianul bisecției are un termen ZZ pentru fiecare pereche de noduri, oricât de rar ar fi graful. Pe 12 noduri: 66 de termeni ZZ, față de 18 la Max-Cut pe un graf 3-regulat. Pe hardware, fiecare termen înseamnă porți cu doi qubiți. De aici vine întrebarea Î5.
- **Alegerea lui A:** dacă muți k noduri dintr-un grup în altul, penalizarea crește cu $4A\cdot k^2$, iar tăietura poate scădea cu cel mult k·d_max (gradul maxim). Ca soluțiile dezechilibrate să nu fie niciodată avantajoase, e suficient A > d_max/4. Alegerea de pornire: A = (d_max + 1)/4. Un A prea mare face „peisajul” greu de optimizat pentru QAOA, deci merită testat.
- **Termenul constant:** nu schimbă care soluție e optimă, dar schimbă energia raportată. Îl păstrezi consecvent peste tot, ca energia Hamiltonianului să fie exact costul problemei.

Unde: Lucas (2014), secțiunea de graph partitioning; tutorialul QAOA IBM, partea „Optimization problem → Hamiltonian”; documentația `SparsePauliOp`.

Verifică-te

- De ce minimizarea lui H_MC maximizează tăietura?
- Câți termeni ZZ are H_B pentru un inel de 10 noduri? Dar H_MC?
- Ce se întâmplă dacă A = 0? Dar dacă A = 1.000?
- De ce verificarea exhaustivă e posibilă aici, dar nu și pe 100 de qubiți?

Task-uri

- [x] ~~Scrie `maxcut_hamiltonian(graph)` și `bisection_hamiltonian(graph, A)`, ambele returnând un `SparsePauliOp` simplificat (`.simplify()`)~~
- [x] ~~Test exhaustiv pentru fiecare instanță: pentru **fiecare** bitstring, energia din Hamiltonian = costul calculat clasic în Etapa 2~~
- [x] ~~Verifică: starea de energie minimă a lui H_B corespunde bisecției optime, iar toate stările de energie minimă sunt echilibrate~~
- [x] ~~Tabel: pentru fiecare instanță, numărul de termeni ZZ în H_MC și în H_B~~

- [x] ~~Pentru 2 instanțe, verifică și pragul lui A: cu A puțin sub d_max/4, apare o soluție dezechilibrată mai „ieftină”? Notează ce observi~~

Livrabil și criteriu de terminare

Modulul `src/hamiltonians.py`, testele în `tests/` și notebook-ul `03_hamiltonieni.ipynb`. Gata când testul exhaustiv trece pe toate instanțele și poți deriva H_B pe hârtie fără să te uiți.

Etapa 4 — QAOA pe simulator (5h)

La finalul etapei ai parametri QAOA optimizați pentru fiecare instanță, problemă și p, plus rezultatele ideale, fără zgomot: plafonul de performanță cu care compari QPU-ul. Răspunde la Î1.

Teorie de învățat

- **Principiul variațional:** energia oricărei stări e cel puțin energia minimă. Deci, dacă ajustezi parametrii unui circuit ca să scadă energia măsurată, te apropii de soluția optimă.
- **Circuitul QAOA:** pornești din superpoziția tuturor bitstring-urilor (H pe fiecare qubit), apoi aplici p straturi. Fiecare strat are un pas de cost (porți RZZ și RZ după termenii Hamiltonianului, cu unghiul γ) și un pas de mixer (RX pe fiecare qubit, cu unghiul β). Total 2p parametri.

```
|\gamma, \beta\rangle = \prod_{k=1}^{p} e^{-i \beta_k H_M} \, e^{-i \gamma_k H_C} \, |+\rangle^{\otimes n}, \qquad H_M = \sum_i X_i
```

- **Intuiția:** pasul de cost dă fiecărui bitstring o fază după costul său; mixerul transformă fazele în probabilități. La p mare, QAOA aproximează evoluția adiabatică și ajunge la optim; la p mic, e o aproximare.
- **Bucla hibridă:** un optimizator clasic (COBYLA, din `scipy`) propune parametri, simulatorul calculează energia, optimizatorul ajustează. Peisajul are minime locale, deci pornești de mai multe ori din puncte aleatorii.
- **Peisajul pentru p = 1:** cu doar doi parametri, poți calcula energia pe o grilă (γ, β) și o desenezi ca hartă. Așa vezi cu ochii tăi unde e minimul și cât de „accidentat” e peisajul. Pentru Max-Cut, $\gamma \in [0, \pi]$ și $\beta \in [0, \pi/2]$ acoperă tot; pentru bisecție, coeficienții diferiți schimbă perioada lui γ, deci explorezi un interval mai larg.

- **Exact vs. cu shot-uri:** simulatorul statevector dă probabilitățile exacte. Cu shot-uri, valorile au eroare statistică de ordinul $1/\sqrt{\text{shots}}$. Folosești varianta exactă la optimizare și varianta cu shot-uri ca să știi cât zgomot statistic să aștepți pe QPU.
- **Transferul parametrilor:** parametrii buni pentru un graf tind să fie buni și pentru grafuri de același tip. Te ajută să eviți optimizarea pe QPU.

Unde: articolul original QAOA (Farhi, Goldstone, Gutmann, 2014), doar introducerea și secțiunea 2; tutorialul QAOA IBM; cursul „Variational algorithm design” de pe IBM Quantum Learning.

Verifică-te

- De ce nu poate energia optimizată să scadă sub costul optim?
- Ce face, intuitiv, pasul de cost și ce face mixerul?
- De ce pornești optimizarea de mai multe ori?
- Ce informație îți dă harta peisajului pe care nu ți-o dă un singur rezultat de optimizare?

Task-uri

- [x] ~~Rulează tutorialul IBM pe graful lui de 5 noduri, complet local (simulator în loc de sesiune), și explică fiecare celulă în jurnal~~
- [x] ~~Scrie `build_qaoa(H, p)` cu `QAOAAnsatz` și `energy(params)` cu `StatevectorEstimator`~~
- [x] ~~Desenează harta peisajului pentru p = 1, pe o grilă 50 × 50, pentru un inel și un graf aleatoriu, Max-Cut și bisecție (4 hărți)~~
- [x] ~~Optimizează pentru fiecare instanță × problemă × $p \in \{1, 2, 3\}$, cu 10 porniri aleatorii, păstrează cei mai buni parametri în `results/params.json`~~
- [x] ~~Pentru fiecare rezultat, din probabilitățile exacte: costul mediu, probabilitatea optimului (suma peste **toate** bitstring-urile optime) și cel mai bun cost dintr-un eșantion de 4.000~~
- [x] ~~Testul de transfer: parametrii de pe inelul de 8 noduri aplicați pe inelul de 12; cât pierzi față de optimizarea directă?~~
- [x] ~~Salvează totul în `results/sim_ideal.csv`, un rând per (instanță, problemă, p)~~

Livrabil și criteriu de terminare

Modulul `src/qaoa.py`, notebook-ul `04_qaoa_simulator.ipynb` cu hărțile de peisaj și graficul „calitate vs. p”. Gata când pe instanțele cu 6 noduri QAOA cu p = 3 dă optimul cu probabilitate mare și poți explica harta de peisaj a unui inel.

Etapa 5 — Transpilare, hardware și predicții (3h)

La finalul etapei știi cât de „scump” e fiecare circuit pe hardware-ul real și ai scris predicții datate despre ce se va întâmpla pe QPU, înainte să rulezi ceva acolo. Pregătește Î2, Î3 și Î5.

Teorie de învățat

- **Porți native:** un QPU execută doar un set mic de porți. Transpilarea rescrie orice altă poartă (RZZ, de exemplu) în porți native.
- **Topologia (coupling map):** pe procesoarele IBM, fiecare qubit e conectat doar cu 2–3 vecini (rețea heavy-hex). O poartă între doi qubiți care nu sunt vecini cere SWAP-uri, iar fiecare SWAP costă trei porți cu doi qubiți.
- **Metricile de cost:** numărul de porți cu doi qubiți și adâncimea circuitului. Porțile cu doi qubiți au erori de ordinul unei zecimi sau al unui procent și sunt sursa dominantă de zgomot.
- **Estimarea succesului:** o estimare grosieră a fidelității circuitului e produsul (1 − eroare) peste toate porțile și citirile, cu erorile luate din calibrarea backend-ului. Un circuit cu sute de porți cu doi qubiți ajunge aproape de zero, adică rezultatul devine practic aleatoriu.
- **Decoerența:** qubiții își pierd starea în timp (T1, T2). Circuitele lungi suferă și din cauza asta.
- **Erori de citire:** uneori un 0 e citit ca 1 și invers. Afectează direct bitstring-urile.
- **Simulatorul cu zgomot:** un *fake backend* din `qiskit_ibm_runtime.fake_provider` copiază topologia și erorile unui QPU real (dintr-un moment fix). Rulează local și gratuit, deci e o repetiție generală înainte de QPU.
- **Predicții înainte de experiment:** scrii ce crezi că se va întâmpla înainte să vezi datele, cu dată. Așa nu-ți poți „ajusta” explicațiile după rezultate. E un obicei de cercetător și se vede bine într-un README.

Unde: ghidurile despre transpilare și despre fake backends din documentația IBM Quantum; pagina backend-ului ales din IBM Quantum Platform (calibrare, topologie).

Verifică-te

- De ce un graf rar poate deveni un circuit adânc pe hardware?
- De ce bisecția are circuite mult mai mari decât Max-Cut pe același graf?
- Ce înseamnă o estimare de succes de 0,05 pentru distribuția pe care o vei măsura?
- Ce poate și ce nu poate să-ți spună un fake backend despre QPU-ul de azi?

Task-uri

- [x] ~~Alege backend-ul țintă și salvează un instantaneu al calibrării (erori, T1/T2) cu dată și oră~~
- [x] ~~Transpilează toate circuitele pentru $p \in \{1, 2\}$ cu `generate_preset_pass_manager(optimization_level=3)`, 5 seed-uri, păstrează varianta cu cele mai puține porți cu doi qubiți~~
- [x] ~~Tabel: instanță, problemă, p, porți cu doi qubiți, adâncime, estimarea succesului~~
- [x] ~~Grafic: porți cu doi qubiți vs. n, câte o curbă pentru Max-Cut și bisecție (Î5)~~
- [x] ~~Rulează pe fake backend un subset (3 instanțe × 2 probleme × p = 1, 2) și calculează aceleași metrici ca în Etapa 4~~
- [x] ~~Scrie în `JURNAL.md`, cu dată, predicțiile pentru QPU: ce instanțe vor merge, care vor fi aproape aleatorii și de ce~~
- [x] ~~Pe baza predicțiilor, fixează lista de circuite pentru QPU (vezi „Matricea de experimente”); include intenționat 2–3 circuite despre care crezi că vor eșua~~

Livrabil și criteriu de terminare

Notebook-ul `05_hardware.ipynb`, fișierul `results/transpiled.csv` și predicțiile în jurnal. Gata când lista pentru QPU e fixată și fiecare predicție are o justificare de o frază.

Etapa 6 — Rulările pe QPU (3h)

La finalul etapei ai toate datele brute de pe hardware real, salvate complet și reproductibil; din acest moment nu mai consumi timp de QPU.

Teorie de învățat

- **PUB-uri:** Sampler primește o listă de „primitive unified blocs”: circuit + valorile parametrilor + număr de shot-uri. Poți trimite zeci de PUB-uri într-un singur job, ceea ce e mult mai eficient decât un job per circuit.

- **Modurile de execuție:** *job* (un job izolat), *batch* (mai multe job-uri împachetate eficient) și *session* (QPU rezervat, pentru bucle iterative). Tu nu ai nevoie de session, fiindcă parametrii sunt deja optimizați. Dacă planul tău permite batch, îl folosești; altfel, câteva job-uri mari.
- **Bugetul:** timpul de QPU depinde în principal de numărul de circuite și de shot-uri. Rulezi întâi un job mic, te uiți în dashboard cât a consumat și abia apoi extrapolezi.
- **Variabilitatea hardware-ului:** calibrarea se schimbă zilnic. Același circuit rulat în zile diferite poate da rezultate diferite, de aceea salvezi calibrarea împreună cu fiecare rulare.
- **Baseline-ul aleatoriu:** bitstring-uri uniform aleatorii, generate clasic, cu același număr de shot-uri. Răspunde la întrebarea „ar fi făcut la fel de bine o monedă?”. Opțional, și un baseline „fizic”: circuitul QAOA cu toți parametrii 0 rulat pe QPU, care dă superpoziția uniformă afectată de zgomotul real.
- **Opțiuni de reducere a erorilor:** Sampler are opțiuni precum dynamical decoupling. Pentru proiectul de bază, le lași la valorile implicite și le notezi; compararea lor e o extensie.

Unde: ghidurile despre primitive, execution modes și monitorizarea job-urilor din documentația IBM Quantum.

Verifică-te

- De ce e mai eficient să pui toate circuitele într-un job decât să trimiți fiecare separat?
- Ce informații trebuie să salvezi ca altcineva să poată reproduce sau verifica rularea?
- Ce ar însemna ca QAOA pe QPU să iasă identic cu baseline-ul aleatoriu?

Task-uri

- [ ] Rulează un job pilot (2 circuite, 1.000 de shot-uri) și notează timpul de QPU consumat; estimează bugetul total
- [ ] Trimite lista fixată în Etapa 5, cu 4.000 de shot-uri per circuit, grupate în cât mai puține job-uri
- [ ] Pentru fiecare job, salvează: ID, backend, data și ora, calibrarea din acel moment, circuitele transpilate (format `qpy`) și rezultatele brute (bitstring-urile), în `data/qpu/`
- [ ] Generează baseline-ul aleatoriu pentru fiecare instanță, cu seed fix
- [ ] Repetă 2–3 circuite reprezentative într-o altă zi, ca să măsori variabilitatea

- [ ] Verifică integritatea datelor: numărul de shot-uri, lungimea bitstring-urilor, corespondența circuit ↔ instanță

Livrabil și criteriu de terminare

Folderul `data/qpu/` complet și notebook-ul `06_qpu.ipynb` care încarcă datele și afișează câte o histogramă. Gata când poți reconstrui orice rezultat din fișierele salvate, fără acces la contul IBM.

Etapa 7 — Analiza și interpretarea datelor (5h)

La finalul etapei ai un răspuns argumentat cu date la fiecare dintre cele cinci întrebări, cu incertitudinile calculate și cu predicțiile comparate cu realitatea. Aici înveți cel mai mult despre interpretarea datelor.

Teorie de învățat

- **Date „tidy”:** un tabel cu un rând per observație (instanță, problemă, p, platformă) și o coloană per variabilă. Orice grafic devine apoi un `groupby` plus un plot.
- **Scorul normalizat:** raportul simplu C/C_opt e înșelător la bisecție, unde optimul poate fi foarte mic (un inel are optimul 2). Folosești un scor care pune pe aceeași scală toate instanțele: 0 = cât media aleatoriului, 1 = optim, negativ = mai rău decât aleatoriul. Aceeași formulă merge și pentru Max-Cut.

```
S = \frac{C_{rand} - C}{C_{rand} - C_{opt}}
```

- **Media vs. cel mai bun din N:** costul mediu al eșantioanelor arată ce face algoritmul „în medie”. Cel mai bun din N arată ce ai folosi în practică, dar crește automat cu N, chiar și pentru aleatoriu. De aceea cel mai bun din N se compară **doar** cu aleatoriul la același N.
- **Factorul de îmbunătățire:** P_QAOA(optim) / P_aleatoriu(optim), unde P_aleatoriu = numărul de bitstring-uri optime / 2^n. La fel pentru fracția de soluții echilibrate (aleatoriu: C(n, n/2)/2^n). Un factor apropiat de 1 înseamnă că QAOA nu a adus nimic.
- **Incertitudinea din shot-uri:** cu 4.000 de shot-uri, orice metrică are zgomot statistic. Bootstrap: reeșantionezi shot-urile cu înlocuire de 1.000 de ori, recalculezi metrica și iei percentilele 2,5 și 97,5 ca interval de încredere de 95%.
- **Variabilitatea între zile:** compari diferența dintre două rulări în zile diferite cu intervalul din bootstrap. Dacă e mult mai mare, hardware-ul variază mai mult decât zgomotul statistic.

- **Corelație vs. cauzalitate:** scorul scade cu numărul de porți, dar și numărul de porți crește cu n. Care e cauza? Comparația Max-Cut vs. bisecție pe **același graf** ține n constant și schimbă doar densitatea circuitului. E un experiment controlat, deci mai convingător.
- **Grafice bune:** fiecare grafic are un singur mesaj, iar titlul e concluzia („Pe QPU, QAOA bate aleatoriul doar până la 8 qubiți”), nu subiectul. Axe cu unități, bare de eroare, aceleași culori pentru aceeași metodă în toate graficele.

Unde: documentația `pandas` (groupby) și `matplotlib`; orice introducere în bootstrap (o pagină e suficientă); capitolele despre grafice din cartea gratuită „Fundamentals of Data Visualization” (Claus Wilke).

Verifică-te

- Ce înseamnă un scor S = −0,1?
- De ce „cel mai bun din 4.000” poate arăta bine chiar dacă QAOA nu funcționează?
- Două rezultate au intervalele de încredere suprapuse în mare parte. Ce poți și ce nu poți concluziona?
- De ce comparația Max-Cut vs. bisecție pe același graf e mai convingătoare decât compararea grafurilor de mărimi diferite?

Task-uri

- [ ] Unifică toate rezultatele (clasic, simulator ideal, fake backend, QPU, aleatoriu) în `results/all.csv`, format tidy
- [ ] Calculează pentru fiecare rând: scorul S (pe costul mediu și pe cel mai bun din N), P(optim), factorul de îmbunătățire, fracția echilibrată (la bisecție)
- [ ] Intervale de încredere bootstrap pentru toate metricile de pe QPU
- [ ] F1 (Î1): scorul ideal vs. p, câte o linie per tip de graf
- [ ] F2 (Î2): scorul vs. n pentru ideal, fake backend și QPU
- [ ] F3 (Î3): factorul de îmbunătățire pe QPU vs. n, cu linia orizontală la 1
- [ ] F4 (Î4): tabel cu toate metodele pe toate instanțele (optim, greedy, QAOA ideal, QAOA QPU, aleatoriu)
- [ ] F5 (Î5): pentru aceleași grafuri, porțile cu doi qubiți și scorul pe QPU, Max-Cut lângă bisecție
- [ ] F6: pentru o singură instanță, histogramele costurilor pentru ideal, QPU și aleatoriu suprapuse; cel mai intuitiv grafic al proiectului

- [ ] F7: scorul pe QPU vs. estimarea succesului din Etapa 5; prezice estimarea rezultatul?
- [ ] Scrie răspunsul la fiecare întrebare în 3–5 propoziții, fiecare cu un număr și o trimitere la un grafic
- [ ] Compară predicțiile din jurnal cu rezultatele: ce ai ghicit, ce nu și de ce
- [ ] Listează limitările (dimensiuni mici, un singur backend, A fix etc.)

Livrabil și criteriu de terminare

Notebook-ul `07_analiza.ipynb`, graficele în `figures/` și documentul `results/concluzii.md`. Gata când fiecare întrebare are un răspuns susținut de un grafic și o incertitudine, iar cineva care citește doar `concluzii.md` înțelege ce ai descoperit.

Etapa 8 — Documentare și prezentare (3h)

La finalul etapei, proiectul poate fi înțeles în 2 minute de un recrutor și reprodus de un alt student.

Teorie de învățat

- **Cine citește un README:** un recrutor citește primele 10 rânduri și se uită la un grafic; un inginer verifică dacă poate rula codul; un specialist caută limitările. README-ul servește toate trei nivelurile, în ordinea asta.
- **Reproductibilitate:** versiuni fixate, seed-uri fixe, date brute salvate și o comandă clară care regenerează graficele.
- **Onestitatea ca punct forte:** o secțiune clară de limitări crește credibilitatea, nu o scade.

Task-uri

- [ ] README cu structura: titlu și o frază de rezumat → graficul principal (F6 sau F3) → problema de sisteme în 3 propoziții → abordarea (diagramă simplă) → rezultatele principale, cu numere → limitări → cum rulezi → structura repo-ului
- [ ] O diagramă a pipeline-ului: graf → Hamiltonian → QAOA → simulator / QPU → analiză, alături de ramura clasică
- [ ] Curăță codul: funcții documentate, fără celule moarte în notebook-uri, testele trec
- [ ] Un script `make_figures.py` care regenerează toate graficele din `data/` și `results/`
- [ ] Completează bullet-ul de CV din secțiunea „La ce proiect lucrezi?” cu cifrele reale

- [ ] Exersează cu voce tare pitch-ul de 30 de secunde și răspunsurile la întrebările de mai jos

Întrebări la care trebuie să poți răspunde la un interviu:

- De ce ai ales QAOA și nu alt algoritm?
- De ce nu bate QAOA forța brută și când ar putea conta asta?
- Ce ai face diferit cu 1.000 de qubiți fără zgomot?
- Care a fost cel mai surprinzător rezultat?
- Ce ai schimba dacă ai relua proiectul?

Livrabil și criteriu de terminare

Repo-ul public pe GitHub. Gata când un coleg poate clona repo-ul, rula `make_figures.py` și obține aceleași grafice, iar tu poți ține pitch-ul fără notițe.

Matricea de experimente

Toate instanțele trec prin solverele clasice și prin simulatorul ideal; pe QPU merge doar un subset de aproximativ 40 de circuite, fixat definitiv în Etapa 5.

Platformă	Instanțe	Probleme	p	Shot-uri
Clasic (forță brută, greedy, aleatoriu)	toate: 4 tipuri × n ∈ {6, 8, 10, 12, 14} + grafurile complete	Max-Cut, bisecție	—	—
Simulator ideal	toate	Max-Cut, bisecție	1, 2, 3	exact + 4.000
Fake backend (cu zgomot)	3 instanțe reprezentative	Max-Cut, bisecție	1, 2	4.000
QPU	inel, 3-regulat, aleatoriu × n ∈ {6, 10, 14}	Max-Cut, bisecție	1, 2	4.000
QPU, repetare în altă zi	3 circuite	cele mai reprezentative	1	4.000

Structura repo-ului

```
qaoa-process-partitioning/
├── README.md
├── JURNAL.md
├── requirements.txt
├── src/
│   ├── graphs.py          # generatoare de instanțe
│   ├── classical.py       # cost, forță brută, greedy, aleatoriu
│   ├── hamiltonians.py    # Max-Cut și bisecție ca SparsePauliOp
│   ├── qaoa.py            # circuit, energie, optimizare
│   └── metrics.py         # scor S, P(optim), bootstrap
├── tests/
├── notebooks/             # 00_setup ... 07_analiza
├── data/
│   ├── instances.json
│   └── qpu/               # rezultate brute, calibrări, circuite qpy
├── results/               # csv-uri, params.json, concluzii.md
├── figures/
└── make_figures.py
```

Riscuri și ce faci

Risc	Cum îl recunoști	Ce faci
Ordinea biților în Qiskit	Energia nu se potrivește cu costul clasic	Testul exhaustiv din Etapa 3, înainte de orice rulare
Optimizarea se blochează	Rezultate foarte diferite între porniri	Mai multe porniri, hărțile de peisaj, A mai mic
Coadă lungă sau buget QPU	Job-ul stă ore în coadă	Trimiți devreme, job pilot, fake backend ca plan B
Rezultatele de pe QPU par aleatorii	Factor de îmbunătățire aproape de 1	Nu e un eșec, e răspunsul la Î3; verifici instanțele mici
API-ul Qiskit s-a schimbat	Erori de import sau parametri	Versiuni fixate, documentația curentă
Depășești timpul	Etapa curentă trece de dublul estimării	Treci pe proiectul minim viabil (mai jos)

Proiectul minim viabil, dacă rămâi fără timp: Etapele 0–4 doar pentru p = 1 și 2, QPU doar pe inel și 3-regulat la n = 6 și 10, analiza doar F3 și F6, README scurt. Rămâne un proiect complet și onest.

Extensii (doar după ce totul e gata, alegi una)

- Comparația cu și fără dynamical decoupling pe QPU
- Date reale de comunicare dintr-un program MPI mic, ca instanțe
- Variația lui A și efectul asupra calității pe simulator
- Un mixer care păstrează echilibrul (XY), fără penalizare, comparat cu varianta ta
- Partiționarea în 4 grupuri prin bisecție recursivă

Resurse

- Tutorialul QAOA, IBM Quantum
- Documentația IBM Quantum și Qiskit
- IBM Quantum Learning: „Basics of quantum information”, „Variational algorithm design”

- Farhi, Goldstone, Gutmann — A Quantum Approximate Optimization Algorithm (2014)
- Lucas — Ising formulations of many NP problems (2014)
- Wilke — Fundamentals of Data Visualization

Glosar

Termen	Pe scurt
QAOA	Algoritm hibrid: un circuit cu parametri, ajustați clasic ca să scadă energia
QUBO	Funcție de cost pătratică în variabile binare
Hamiltonian	Operatorul a cărui „energie” e costul problemei
p	Numărul de straturi (cost + mixer) din QAOA
Shot	O singură rulare și măsurare a circuitului
Transpilare	Rescrierea circuitului în porțile și topologia unui QPU anume
SWAP	Poartă care schimbă stările a doi qubiți; costă 3 porți cu doi qubiți
Fake backend	Simulator care imită zgomotul și topologia unui QPU real
Scor S	0 = cât aleatoriul, 1 = optim
Bootstrap	Reeșantionare a datelor pentru intervale de încredere